

SDD

概觀

開發環境建立

建立開發環境

環境建立(for-vscode+spekit)

環境檢核 specify check

`specify check`

`specify check` 階段會檢查系統中是否已安裝並可使用各種開發工具，例如Git、GitHub Copilot、Claude Code等。

REPO建立

初始化specify框架 specify init

`specify init --ai copilot --script ps --here`

這個指令會初始化一個Specify專案，設定AI助理為Copilot、腳本類型為PowerShell，並下載專案範本、解壓縮，最後初始化Git儲存庫，讓專案準備好開始使用。

專案自訂規則初始化

舉例 .github\copilot-instructions.md

一律使用 zh-tw 溝通

SDD with spekit

制定憲法 spekit.constitution

`/spekit.constitution` 【其他補充描述】

整個專案的最高原則

產出

- 建立憲章文件
 - 原文：constitution
 - 雙語：constitution_雙語對照

制定規格 spekit.specify

`/spekit.specify` @需求描述.md

描述需要實現的功能或者需要規節的問題

產出

- 建立分支：001-XXX
- 建立 spec.md 文件
 - specs\001-XXX\spec.md

制定計劃 spekit.plan

`/spekit.plan` @工程開發計畫描述.md

產出

specs\001-XXX\plan.md
specs\001-XXX\data-model.md
specs\001-XXX\quickstart.md
specs\001-XXX\research.md

規劃任務清單 spekit.task

`/spekit.task`

產出

specs\001-XXX\task.md

開始實作 spekit.implement

`/spekit.implement`

建立相關代碼與文件

建立相關代碼與文件

```
D:\
├─src
│   ├──DoraemonGadgets.App
│   ├──DoraemonGadgets.Core
│   ├──DoraemonGadgets.Infrastructure
│   └─DoraemonGadgets.Telemetry
└─tests
    ├──contract
    ├──integration
    └─unit
```

環境預備

套件安裝

參見 [環境建立\(for-vscode+speckit\)](#)

REPO

1. 建立REPO，開啟VS CODE。

2. cmd 初始化 speckit

```
specify init --ai copilot --script ps --here
```

3. 建立 `copilot-instructions.md`

需求描述

prompt-spec.md

<https://github.com/agclct77/Rivet/blob/master/prompt-spec.md>

```
1  # 產品名稱：
2  Rivet
3
4  # 目的
5  我的工具箱
6
7  # 功能需求：
8  ## 主訴
9  按下快捷鍵(Alt+.)會出現選單
10  選單內容：[1]翻譯成繁體中文 [2]翻譯成英文
11  選擇1之後將剪貼簿中的資料翻譯成指定語言
12  知會用戶已經將譯文放到剪貼簿中
13  ## 其他案例
14  - 如果剪貼簿內容非CF_UNICODETEXT，予以回饋： 無法翻譯非純文字
15  - 如果剪貼簿內容CF_UNICODETEXT為空，予以回饋： 剪貼簿內容為空無法翻譯
16
17
18  # 結構要求：
19  *至少會有三個*
20  - Rivet.Console：前端，之後有可能會換成WPF或其他
21  - Rivet.Service：邏輯處理
22  - Rivet.Infrastructure：基礎設施
23
24  # 框架技術堆棧
25  - NET 8
26  - Spectre.Console
```

```
27 - Google Cloud Translation API
28 - Serilog
29 - Nunit+Nsubstitute
30
31 # 非功能性要求
32
33 ## 擴充性/變動性
34 - 保留可能會替換前端技術
35 - 保留可能會替換翻譯技術
36 - 功能選單會增加，以責任鏈模式匹配指令，未來功能增加詞，不改動既有代碼。
37
38 ## 其他
39 - 剪貼簿內容需要輸出trace層級方便debug。
40 - log輸出到 %AppData% 下
```

開發過程

1. speckit.constitution

GPT-5.1-Codex + speckit.constitution

```
1 1. Create principles focused on code quality, testing standards, user
  experience consistency, and performance requirements.
2 2. All specifications, plans, and user-facing documentation MUST be written
  in Traditional Chinese (zh-TW), except for this constitution document
  itself.
3 3. Fellow Git Commit Message Convention
4   1. Standard & Language:
5     - Strictly follow the Conventional Commits specification.
6     - All descriptions (subject and body) must be written in
  Traditional Chinese (Zh-TW).
7   2. Header Format:
8     - The header structure must be: `<type>(<Ticket-ID>): <description>`
9     - Type: Use standard types (e.g., `feat`, `fix`, `docs`,
  `style`, `refactor`, `chore`, `ci`).
10    - Scope: The `<Ticket-ID>` (e.g., DG-2, TOOL-1) must be placed
  inside the parentheses as the scope.
11    - Description: A concise summary of the change in Zh-TW.
12    - Example: `feat(DG-2): 實作全域熱鍵監聽功能`
```

Summary

Constitution 建立完成

新版本: 1.0.0

版本類型: Initial Release (首次建立)

已建立的原則 (6 項)

原則編號	名稱	說明
I	Code Quality	程式碼品質標準 (可讀性、可維護性、一致性、錯誤處理)
II	Testing Standards	測試標準 (80% 覆蓋率、單元/整合/契約測試)
III	User Experience Consistency	使用者體驗一致性 (設計語言、無障礙、回饋機制)
IV	Performance Requirements	效能需求 (回應時間、記憶體效率、資源管理)
V	Language Policy	語言政策 (所有 spec/plan/tasks 必須使用正體中文)
VI	Git Commit Convention	Git Commit 規範 (Conventional Commits + Ticket ID + 正體中文)

模板相容性檢查

- **plan-template.md**:  相容 (Constitution Check 區段已存在)
- **spec-template.md**:  相容 (Requirements 區段已對齊)
- **tasks-template.md**:  相容 (Phase 結構已對齊)

待追蹤事項

無待處理項目。

2. speckit.specify

GPT-5.1-Codex + speckit.specify + prompt-spec.md

1 `do it`

Summary

 **規格建立完成**

項目	值
功能分支	<code>001-clipboard-translator</code>

項目	值
規格檔案	spec.md
檢查清單	requirements.md
狀態	所有驗證項目通過 

規格摘要

功能名稱: 剪貼簿翻譯工具

User Stories (共 5 個):

- **P1:** 翻譯成繁體中文、翻譯成英文、快捷鍵呼叫功能選單
- **P2:** 處理非純文字內容、處理空白剪貼簿

Functional Requirements (共 10 項):

- 全域快捷鍵 Alt+. 呼叫選單
- 支援翻譯成繁體中文/英文兩個選項
- 剪貼簿讀寫與錯誤處理
- Trace 層級日誌記錄
- 責任鏈模式支援未來擴充

Success Criteria (共 5 項):

- 選單顯示時間 $\leq 500\text{ms}$
- 翻譯操作 ≤ 5 秒 (500 字元內)
- 首次使用 30 秒內完成翻譯
- 所有錯誤都有友善訊息
- 背景記憶體佔用 $\leq 100\text{MB}$

下一步

規格已準備就緒，可執行 `/speckit.plan` 進行技術規劃。

GPT-5.1-Codex

- 2 每一次按下快捷鍵都是新開實例
- 3 使用者可以用數字件來選擇功能

Summary

規格更新完成

變更摘要

項目	變更前	變更後
支援語言	多語言（日文、簡中等）	僅繁中↔英文雙向
執行模式	背景執行	每次新開實例
選單操作	選擇選單項目	數字鍵 1/2 快速選擇

新增/修改的需求

- **FR-001**: 更新為「開啟新的應用程式實例並顯示功能選單」
- **FR-002**: 加入「使用者可按對應數字鍵選擇」
- **FR-011** (新增): 系統僅支援繁體中文與英文之間的雙向翻譯
- **FR-012** (新增): 翻譯完成並通知使用者後，應用程式實例必須自動結束

更新的 Success Criteria

- **SC-005**: 從「背景記憶體 ≤ 100MB」改為「實例執行時間不超過 10 秒」

更新的 Edge Cases

- 新增：使用者按下非 1、2 的數字鍵時如何處理
- 更新：連續按快捷鍵改為「每次都開啟新實例，各實例獨立運作」

Out of Scope 新增項目

- 繁體中文與英文以外的語言翻譯
 - 常駐背景執行模式
-

規格已準備就緒，可執行 `/speckit.plan` 進行技術規劃。

GPT-5.1-Codex + speckit.clarify

1 do it

 Note

問題 1/5

關於全域快捷鍵的實現方式：

規格提到「Alt+. 快捷鍵在使用者系統上未被其他應用程式佔用（需額外設定系統快捷鍵）」。這涉及到快捷鍵如何被觸發以開啟 Rivet 實例。

Recommended: Option B - 使用 Windows 內建的「鍵盤快速鍵」功能設定捷徑檔 (.lnk) 是最簡單且不需要額外程式碼的方式，使用者可自行在桌面捷徑屬性中設定快捷鍵。

Option	Description
A	開發獨立的背景服務監聽全域快捷鍵並啟動 Rivet 實例
B	使用 Windows 捷徑檔 (.lnk) 的「快速鍵」屬性，由使用者自行設定
C	使用 Windows Task Scheduler 搭配快捷鍵觸發任務
Short	提供不同的簡短答案 (≤5 words)

您可以回覆選項字母（例如「A」）、接受建議選項說「yes」或「recommended」，或提供您自己的簡短答案。

- 1 我同意將規格簡化為使用方案B
- 2 但由於方案B的特性，我要將快捷鍵由`Alt+.`改為`Ctrl+Alt+.`

 Summary

 澄清流程完成

提問統計

項目	數量
提問數	5
已回答	5

更新的規格檔案

spec.md

變更的區段

區段	變更內容
Clarifications	新增 5 個澄清問答記錄
User Scenarios	快捷鍵更新為 Ctrl+Alt+.
Functional Requirements	FR-001, FR-002, FR-005 更新；新增 FR-013, FR-014
Assumptions	更新快捷鍵設定與認證方式說明
Edge Cases	更新無效輸入處理（含 Esc 取消）

覆蓋摘要表

類別	狀態
功能範圍與行為	✔ Resolved
範圍外聲明	✔ Clear
使用者角色區分	✔ Clear
領域與資料模型	✔ Clear
互動與 UX 流程	✔ Resolved
非功能性品質屬性	✔ Clear
整合與外部依賴	✔ Resolved
Edge Cases 與失敗處理	✔ Resolved
限制與取捨	✔ Clear
術語一致性	✔ Clear
完成信號	✔ Clear

所有關鍵歧義已解決，無 Outstanding 或 Deferred 項目。

建議下一步

執行 `/speckit.plan` 進行技術實作規劃。

3. speckit.plan

Claude Opus 4.5 + speckit.plan + prompt-spec.md

1 do it

 Note

 規劃完成報告

分支資訊

- **Branch:** 001-clipboard-translator
- **IMPL_PLAN:** plan.md

產出文件

檔案	說明
plan.md	✓ 實作計劃（含技術上下文、架構設計、Constitution Chec...
research.md	✓ Phase 0 研究文件（技術決策與最佳實踐）
data-model.md	✓ Phase 1 資料模型（5 個核心實體定義）
interfaces.md	✓ Phase 1 介面合約（5 個服務介面）
quickstart.md	✓ Phase 1 快速入門指南
copilot-instructions.md	✓ Agent context 已更新

Constitution Check 結果

- **Pre-Phase 0 Gate:** ✓ 通過
- **Post-Design Gate:** ✓ 通過

下一步

執行 `/speckit.tasks` 產生任務清單（tasks.md），進入 Phase 2 實作階段。

4. speckit.task

Claude Opus 4.5 + speckit.task

Summary

完成摘要

已生成 `tasks.md`

統計資訊

項目	數量
總任務數	50 個
Phase 1: Setup	7 個任務
Phase 2: Foundational	21 個任務
Phase 3: US3 (選單)	4 個任務
Phase 4: US1 (英翻中)	3 個任務
Phase 5: US2 (中翻英)	3 個任務
Phase 6: US4 (非純文字處理)	2 個任務
Phase 7: US5 (空白處理)	2 個任務
Phase 8: Polish	8 個任務

平行執行機會

- **Phase 1:** T002-T004 可平行 (3 個專案建立)、T006-T007 可平行
- **Phase 2:** T009-T017 可平行 (所有 models)、T018-T022 可平行 (所有介面)、T024-T028 可平行 (基礎設施實作)
- **Phase 4 & 5:** User Story 1 和 2 可平行進行
- **Phase 6 & 7:** User Story 4 和 5 可平行進行

獨立測試標準

User Story	獨立測試方式
US3 (選單)	執行程式，驗證顯示選單，按 Esc 可關閉
US1 (英翻中)	複製英文，按 1，驗證剪貼簿為繁中
US2 (中翻英)	複製繁中，按 2，驗證剪貼簿為英文

User Story	獨立測試方式
US4 (非純文字)	複製圖片，選擇翻譯，驗證錯誤訊息
US5 (空白)	清空剪貼簿，選擇翻譯，驗證錯誤訊息

建議 MVP 範圍

Phase 1-4 (Setup + Foundational + US3 + US1)：使用者可執行程式、看到選單、將英文翻譯成繁體中文。

格式驗證

所有 50 個任務皆遵循 checklist 格式：

-  包含 checkbox (- [])
-  包含 Task ID (T001-T050)
-  有 [P] 標記表示可平行任務
-  User Story phase 任務包含 [US1]-[US5] 標籤
-  包含具體檔案路徑

5. speckit.analyze

Claude Opus 4.5 + `*speckit.analyze`

- 1 請將分析報告輸出到 分支目錄底下，方便以後追溯。

Note

分析報告已建立完成！

已載入的文件

- `tasks.md` - 任務清單 (51 項任務)
- `spec.md` - 功能規格
- `plan.md` - 實作計畫
- `constitution.md` - 專案規範
- `data-model.md` - 資料模型
- `interfaces.md` - 介面合約
- `quickstart.md` - 快速入門指南



分析報告摘要

報告已輸出至：`analysis-report.md`

結果概覽

嚴重性	數量
Critical	0
High	2
Medium	5
Low	4

主要發現

1. **[HIGH] A1** - `CommandHandlerBase` 在 `tasks.md` 有任務但 `contracts` 未定義合約
2. **[HIGH] A2** - 缺少測試專案建立任務（違反 Constitution II 測試標準）
3. **[MEDIUM] B3** - `ICommand.cs` 在 `plan.md` 出現但無對應任務或合約定義

評估結論

✅ **可進入實作階段** - 無 CRITICAL 問題阻擋，所有功能需求與 User Stories 皆有對應任務覆蓋（100%）

是否需要我針對 HIGH 優先級問題提供具體的修改建議？

6. speckit.implement(Phase1)

Claude Haiku+ `speckit.implement*`

- 1 請依據 `#file:tasks.md` 依序進行實作，下一階段：**Phase 1**
- 2 - 每完成一個任務(**T, Task**)，應保證程式可以建置成功且測試通過後才能進入下一個任務；如果建置或測試失敗，應立即修正。
- 3 - 每完成一個階段(**Phase**)
- 4 - 應該生成以 `zh-tw` 撰寫的提交訊息，給我審閱。
- 5 - 如果我確認通過，我自行完成 `git` 提交，再請你進行下一階段；反之我會說明這個階段還需要修改什麼。
- 6 - 需要更新 `#file:tasks.md`，反應實際進度。
- 7 - 過程中如果有問題，請建議相關處理措施，並自主實作最佳建議方案，無需詢問我。

Summary

Phase 1 & 1.5 完成總結

已完成任務：

-  T001-T007: 建立方案檔與三個核心專案
-  T007a-T007d: 建立測試專案與安裝測試框架

項目結構已建立：

略

構建狀態： 全部成功

- 編譯：成功 (3 個核心專案)
- 測試建置：成功 (2 個測試專案)
- 測試執行：成功 (2/2 測試通過)

提交訊息建議：

略

請確認提交訊息無誤後告訴我，我會繼續進行 Phase 2 (Foundational) 的實作！

repeat 8 次(8個階段)

注意事項

- 注意上下文長度。每告一個段落，應清理工作環境，確認哪些資料應落地保存。然後再開啟新對話。
- prompt太長就另開文檔撰寫，以附檔的方式載入對話視窗。例如 `prompt-spec.md`。

附錄

跟本專案有關的

- REPO：<https://github.com/agclct77/Rivet>

無關的

- https://microsoftlearning.github.io/mslearn-github-copilot-dev/Instructions/Labs/LAB_AK_06_vibe_coding_prototype_ecommerce_app.html

